Analyzing the Development Lifecycle of Vichan via Automated Metadata Classification

COSC 445 Final Project Proposal · Tristan Lopez, Noah Cash, Jonathan Lange, Ar-Raniry Ar-Rasyid
Repository: https://github.com/jdlange3/COSC445-Final-Project

Objective

Our goal is to mine and analyze the development history of **Vichan**, a mid-sized open-source imageboard engine, and of the forks it sits between (Tinyboard before it; Infinity and Lainchan after it). We will extract commit metadata and code diffs, classify every change by its purpose, and use those labels to reconstruct how the project's priorities shifted over its 16-year life (2010–2026). We are trying to answer two questions:

- **Did Vichan move from feature development to mostly security and maintenance work as it matured**, and when did that shift happen?
- **How much of the code churn came from outside pressure** (security vulnerabilities, PHP version deprecations, spam and abuse) compared with internal work such as refactoring?

The result will be a labeled, deduplicated dataset of the Vichan lineage and a timeline that lines up category spikes with known events in the project's history (the Tinyboard fork, the 2013 peak, the 2018–2020 near-dormancy, the 2024 revival, and the 2026 end-of-life notice).

Motivation

Most open-source projects are small, community-run, and eventually abandoned, but there is little data on how their work changes as they age. Vichan is a good case study for Digital Archaeology because its history is complete, public, and finished: the repository was archived in July 2026 with a README declaring the project's end of life. It also sits in the middle of a fork lineage whose branches were driven by real-world events, so the commit log is a record of how a community responded to outside pressure over time.

Reading roughly ten thousand commits by hand is slow and inconsistent. Automated classification (keyword heuristics backed by LLM-assisted labeling) lets us turn short, unstructured commit messages and diffs into quantitative data, and checking those labels against a hand-coded sample tells us how far the method can be trusted.

Data

Sources. (1) The Git histories of four related repositories: *vichan-devel/vichan* (all branches), *savetheinternet/Tinyboard*, *ctrlcctrlv/infinity*, and *lainchan/lainchan*. (2) The GitHub REST API for *vichan-devel/vichan* issues, pull requests, and their comments, which give us the context that commit messages leave out.

Retrieval. Repository mining: a Python script (GitPython / `git log` subprocess wrappers) clones each repository as a bare mirror, fetches all of them into one object database so shared history is deduplicated by commit SHA, and writes structured CSVs. A second script pages through the GitHub API (authenticated, 5,000 requests/hour) for issue and PR records.

Estimated volume (measured from preliminary clones on 2026-10-22):

Record type	Unit	Approx. count
Commits (unique across the four repositories)	commit	10,189 (8,862 non-merge)
File-level changes	commit × file	27,404
Diff hunks (unit of change classification)	hunk	68,957

GitHub issues and pull requests (Vichan)	issue / PR	~950, plus comment threads
Distinct author emails (before alias merging)	identity	259

Overall we expect to process about 100,000 records, with diff hunks (~69,000) as the main unit of analysis.

Expected challenges.

- **Incomplete context:** many messages are terse ("update", "fix"), so we will classify using the diff and changed file paths as well as the message, and pull in linked PR/issue text where it exists.
- **Inconsistent identities:** one person committed under at least six name/email combinations (e.g. "Savetheinternet", "Michael Foster", "Fredrick Brennan"), and web merges list GitHub as the committer. We need to resolve aliases before any per-person analysis.
- **Overlapping forks:** the four repositories share early history, so records must be deduplicated by SHA and each commit assigned to the lineage where it first appears.
- **Unreliable timestamps and rewritten history:** dates are set by the committer's machine and history may have been squashed or rebased. We will compare author and committer dates and flag commits that are out of order.
- **Label validity:** automatic labels can be wrong, so we will measure accuracy against a hand-labeled sample.

Approach

The project covers **retrieval, storage, cleaning, context recovery, and analysis**.

- **Retrieval and storage:** scripted cloning and extraction into three linked tables (commits, file changes, hunks) keyed by SHA, plus API tables for issues and PRs. All of it goes into a SQLite database so it can be queried again.
- **Cleaning and normalization:** separate merge commits, convert timestamps to UTC, deduplicate shared history across forks, build a `.mailmap`-style alias table using email and name matching with manual review, and flag bot or automated commits.
- **Classification:** a two-stage pipeline. Stage one uses keyword and file-path rules (for example CVE/XSS/CSRF/injection terms mean Security; `*.md` files mean Documentation). Stage two sends the remaining or low-confidence records to an LLM with the message, file list, and a truncated diff. Each commit is tagged *Feature, Bugfix, Refactoring, Security, Documentation,* or *Maintenance/Dependency*.
- **Validation:** two team members independently hand-label a stratified random sample of about 400 commits. We report agreement between the coders (Cohen's κ) and the pipeline's precision and recall for each category.
- **Context recovery and analysis:** aggregate labels by month and by era, test whether the Security and Maintenance share rises over time, measure churn (lines changed) by category, match security spikes to linked issues and PRs, and compare category mixes across the four forks.

Tools: Python, Git, GitPython, pandas, SQLite, the GitHub REST API, an LLM API, and Matplotlib/Seaborn.

Team Responsibilities

Member	Role	Responsibilities
Jonathan Lange	Data Engineer	Write the cloning and extraction scripts and the GitHub API collector, define the CSV/SQLite schema, and validate the raw data (record counts, missing fields).

Tristan Lopez	Cleaning & Integration	Deduplicate across forks, handle merges and timezones, resolve author aliases, flag bots, and link commits to issues and PRs.
Ar-Raniry Ar-Rasyid	Classification Lead	Write the keyword rules and LLM prompts, run the full classification, and coordinate the hand-labeled validation sample and accuracy metrics.
Noah Cash	Analysis & Reporting	Run the timeline and churn analysis, make the visualizations, lead the final report and presentation, and match spikes to historical events.

Every member hand-labels part of the validation sample and contributes to the final report.

Timeline and Milestones

Week of	Milestone	Deliverable
Oct 26	Data retrieval	Extraction scripts for all four repositories; raw commit, file, and hunk CSVs; issue and PR data from the API.
Nov 2	Cleaning & storage	Deduplicated SQLite database, alias table, merge and bot flags, UTC timestamps.
Nov 9	Pipeline setup	Rules and LLM prompts tested on a 200-commit pilot; codebook written; validation sample drawn.
Nov 16	Full classification	Whole dataset labeled; hand-labeling done; accuracy and κ reported; prompts revised if needed.
Nov 23	Analysis	Monthly and per-era timelines, churn by category, fork comparison, spikes matched to events.
Nov 30	Final report	Written report, final figures, cleaned code and data in the repository, presentation.

Expected Outcome

The main result is a **data analysis** with a clear finding: a validated, category-labeled timeline of the Vichan lineage that shows whether and when the project shifted from feature work to security and maintenance work, and how much of its churn came from outside pressure versus internal refactoring. Supporting outputs are (1) a reusable, documented pipeline that extracts, deduplicates, and classifies commits from any Git repository, (2) a cleaned SQLite dataset of about 100,000 linked records with resolved author identities, and (3) an accuracy assessment of automated commit classification on a real, messy history. If the hypothesis is not supported, for example if Security never becomes the dominant category, that is still a result the data can back up, and we will report it.